Principes de DevOps

2IE / S7 IA & INFO, 2025 / 2026

Cours dispensé par :
Wend N'Contet Latiifah Rosemond
Consultante en Business Intelligence & Analytics / IA
latiifahsoulga@gmail.com

Table des matières

Ingénierie DevOps : Principes, Pratiques et Implémentation 3
Introduction à DevOps 5
1. Introduction à DevOps 5
   a. Définition 5
   b. Origine et histoire 5
2. Objectifs et avantages de DevOps 7
   a. Une collaboration accrue 7
   b. Des cycles de livraison plus rapides 8
   c. amélioration continue 10
Principes de Base de DevOps 12
1. La culture DevOps : ce que cela signifie réellement 12
   a. Les valeurs fondamentales de DevOps 12
   b. Collaboration et communication : un changement de posture 13
   c. L'impact sur les processus et les performances 14
2. Cycle de Vie DevOps Introduction aux Outils DevOps 16
   a. Les grandes étapes du cycle DevOps 16
   b. Une boucle continue, pas une ligne droite 17
   c. Ce que DevOps change réellement dans le cycle de vie 18
Introduction aux Outils DevOps 19
1. Jenkins pour CI/CD 19
   a. Pourquoi Jenkins (et pourquoi la CI/CD) ? 19
   b. Définition : Intégration Continue (CI) 20
   c. Définition : Livraison Continue (CD) 20
   d. Jenkins : rôle et fonctionnement 21
   e. Notion de pipeline 21
   f. Bonnes pratiques CI/CD avec Jenkins 21
2. Docker et la gestion des conteneurs 22
   a. Pourquoi Docker est devenu central en DevOps ? 23
   b. Conteneur vs Machine virtuelle (VM) 23
   c. Concepts fondamentaux Docker 24
   d. Avantages Docker dans DevOps 24
   e. Docker dans un pipeline DevOps 25
3. Kubernetes pour l'orchestration des conteneurs 25

a. Pourquoi Kubernetes ? 25
b. Concepts clés Kubernetes 26
c. Déploiement d'une application 26
d. Pourquoi Kubernetes est un outil DevOps clé ? 27
4. Ansible pour la gestion des configurations 28
a. Le problème : la configuration manuelle 28
b. Ansible : définition 28
c. Concepts fondamentaux Ansible 29
d. Ansible dans une démarche DevOps 29
Automatisation des Processus DevOps 30
Partie 1 - Scripts d'automatisation & Infrastructure as Code (Terraform) 30
1. L'automatisation : le pilier technique de DevOps 30
2. Pourquoi les tâches manuelles deviennent un problème à grande échelle 31
3. Scripts DevOps : Bash et Python 32
4. Bash vs Python : quand utiliser quoi ? 33
5. Bonnes pratiques essentielles pour les scripts DevOps 33
6. Infrastructure as Code (IaC) : automatiser l'infrastructure 34
7. Terraform : introduction et logique de fonctionnement 35
8. Ce que Terraform apporte concrètement en DevOps 37
Partie 2 - Orchestration avancée Kubernetes & Déploiement automatisé Ansible 38
1. Kubernetes avancé : pourquoi c'est important ? 38
2. Orchestration : ce que Kubernetes fait réellement 38
3. Notions avancées Kubernetes (bien expliquées) 38
4. Ansible : automatiser le déploiement dans la vraie vie 40
5. Ansible avancé : structurer pour être maintenable 41
6. Vision DevOps : connecter Jenkins, Docker, Kubernetes, Ansible 42
Synthèse générale — Automatisation DevOps 43
Sécurité dans DevOps 44
Partie 1 - DevSecOps : intégrer la sécurité dès le début & outils de sécurité 44
1. Pourquoi la sécurité est devenue un enjeu central en DevOps 44
2. Le problème réel : la sécurité arrive trop tard 45
3. DevSecOps : définition et philosophie 45
4. Le principe "Shift Left" : déplacer la sécurité vers la gauche 46
5. Quels sont les risques typiques dans un environnement DevOps ? 46

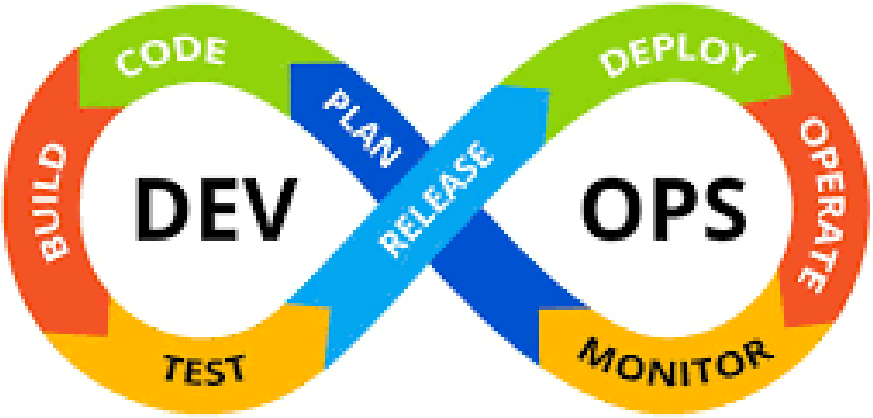
6. Outils de sécurité dans DevOps : comprendre les familles d'outils 48
7. Intégration des outils de sécurité dans Jenkins 50
Partie 2 - Sécurité Kubernetes, gestion des accès, chiffrement & surveillance 51
1. Pourquoi Kubernetes demande une sécurité spécifique ? 51
2. Sécurité Kubernetes : le principe du moindre privilège 51
3. RBAC : contrôle des accès par rôles 52
4. Politiques de sécurité Kubernetes 53
5. Gestion des secrets 54
6. Chiffrement et surveillance : sécurité + résilience 54
7. Pourquoi DevOps + sécurité = un équilibre 55
Synthèse générale — Sécurité dans DevOps 56

Ingénierie DevOps : Principes, Pratiques et Implémentation

Introduction : La Transformation Fondamentale de la Production Logicielle

L'industrie du développement logiciel traverse une mutation sans précédent, redéfinissant non seulement les outils techniques utilisés, mais également la structure même des organisations et la culture de collaboration. Ce cours, conçu comme un manuel de référence exhaustif pour le module "Principes de DevOps", vise à fournir une compréhension encyclopédique et pratique de ce paradigme. Il s'adresse à des étudiants possédant un bagage informatique généraliste mais débutant dans les réalités opérationnelles de la mise en production. L'objectif est de combler le fossé entre la théorie académique et l'exigence industrielle, en explorant chaque concept avec une granularité fine, soutenue par des exemples techniques concrets et une analyse rigoureuse des mécanismes sous-jacents.

Introduction à DevOps



1. Introduction à DevOps

a. Définition

DevOps est une culture et un ensemble de pratiques qui favorisent la collaboration entre :

- ✓ les équipes de **développement logiciel (Dev)**
- ✓ et les équipes d'**exploitation / infrastructure (Ops)**

Le but est de réduire les barrières entre ces équipes afin de livrer des applications :

- ✓ plus rapidement,
- ✓ plus souvent,
- ✓ avec plus de fiabilité.

b. Origine et histoire

DevOps est né en réponse à un problème classique :
les développeurs produisent des applications, mais les équipes système doivent ensuite les déployer et les maintenir… souvent dans l'urgence, sans informations suffisantes, et avec des contraintes de sécurité et de disponibilité.

Cette séparation a longtemps provoqué :

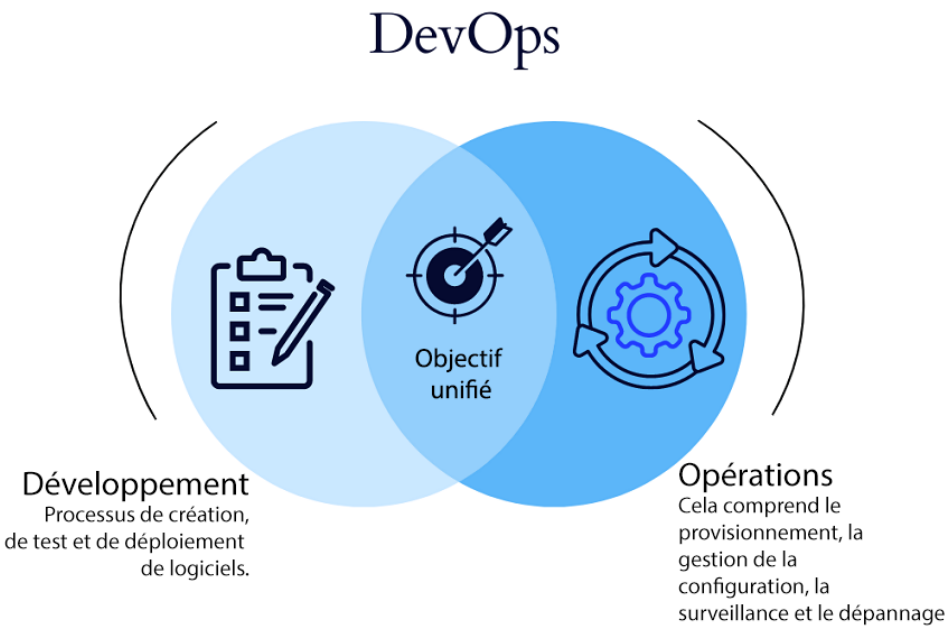
- des conflits (“ça marche chez moi”, “ça marche pas en prod”),
- des retards de mise en production,
- des déploiements risqués,
- et une baisse de qualité globale.

DevOps apparaît donc comme une réponse pragmatique :
***« faire travailler ensemble les équipes qui construisent le logiciel et celles qui l’exploitent**. »*

Différences avec les méthodes traditionnelles : Waterfall et Agile

- **Waterfall (cycle en cascade)**
  - Les étapes sont strictement séparées : analyse → conception → développement → tests → production
  - Avantage : clair et structuré
  - Limite : lent, rigide, les retours arrivent trop tard
- **Agile**
  - Les développements se font par petites itérations (sprints)
  - Avantage : plus flexible, plus rapide, feedback continu
  - Limite : Agile améliore surtout le travail côté développement, mais **ne résout pas totalement la partie déploiement et exploitation**
- **DevOps**
  - DevOps complète Agile en intégrant la dimension **déploiement, infrastructure, supervision, sécurité, automatisation**
  - DevOps s’intéresse au cycle complet : du code jusqu’à l’application en production et sa surveillance.

2. Objectifs et avantages de DevOps



a. Une collaboration accrue

DevOps, ce n'est pas une baguette magique. C'est un changement de mentalité. Historiquement, les équipes "Dev" (développement) et "Ops" (opérations) travaillaient presque comme deux camps opposés. Les développeurs voulaient aller vite, ajouter des fonctionnalités. Les opérations voulaient de la stabilité, éviter les pannes. Résultat ? Frictions constantes.

Quand on parle de collaboration accrue, on parle d'aligner ces deux logiques au lieu de les opposer.

- Objectifs partagés

Avant, chacun avait son propre indicateur de succès. Le développeur était jugé sur la rapidité et le nombre de fonctionnalités livrées. L'équipe Ops sur la stabilité et l'absence d'incidents.
DevOps change ça : tout le monde est responsable du même objectif final — livrer un produit stable, performant et utile aux utilisateurs.
Ça change profondément le comportement. Si ton code provoque une panne en production, ce n'est plus "le problème des Ops". C'est ton problème aussi.

- Communication fluide

Dans un modèle classique, les échanges passent par des tickets, des validations, des couches hiérarchiques. Beaucoup d'énergie se perd dans la friction.
Avec DevOps, on favorise les échanges directs, les réunions communes, les outils partagés (comme l'intégration continue, les pipelines automatisés). On remplace le "je te passe le problème" par "on résout ça ensemble".
C'est un peu comme passer d'un jeu en tour par tour à un jeu coopératif en temps réel.

- Responsabilités plus claires

DevOps ne signifie pas que tout le monde fait tout. Au contraire. Les rôles restent spécialisés, mais la responsabilité est collective.
Un développeur doit comprendre comment son application tourne en production. Un Ops doit comprendre les contraintes du code.
On réduit ainsi les zones floues où chacun pouvait dire : "ce n'est pas moi".

- Réduction des tensions

Les tensions viennent souvent d'objectifs contradictoires et d'un manque de compréhension mutuelle.
Quand les équipes partagent les mêmes indicateurs (temps de déploiement, taux d'erreurs, satisfaction utilisateur), le conflit structurel disparaît.
On passe d'un rapport presque "tribal" à une logique d'équipe produit unifiée.

Si on veut résumer : DevOps transforme une frontière en pont.

Et il y a un détail fascinant là-dedans : ce n'est pas d'abord un changement technique, mais culturel. Les outils (automatisation, CI/CD, monitoring) ne sont que des accélérateurs. Le cœur du système, c'est la coopération humaine. Et comme souvent en ingénierie, le vrai bug n'est pas dans la machine, mais dans l'organisation.

b. Des cycles de livraison plus rapides

Cette fois, on parle de vitesse. Pas la vitesse "on code comme des fous", mais la vitesse intelligente.

Avant DevOps, livrer une nouvelle version ressemblait souvent à une opération chirurgicale à haut risque. On accumulait des changements pendant des semaines, puis on faisait un "gros déploiement". Forcément, plus c'est gros, plus c'est fragile. Une seule erreur et tout casse.

DevOps change la stratégie : au lieu de livrer rarement et massivement, on livre souvent et en petites quantités. C'est presque une logique biologique. Les petites mutations sont plus faciles à absorber que les grosses secousses.

- Livrer plus fréquemment

Quand les processus sont automatisés, on peut déployer plusieurs fois par jour si nécessaire. Chaque modification est petite, donc plus facile à tester et à corriger. Le risque global diminue, même si la fréquence augmente. C'est contre-intuitif, mais c'est vérifié dans la pratique.

- Corriger plus vite les bugs

Si un bug apparaît, on peut pousser un correctif rapidement. On ne doit plus attendre "la prochaine grosse version". Le délai entre la détection et la correction se réduit drastiquement. En ingénierie, on appelle ça réduire le "lead time" : le temps entre l'idée (ou le problème) et la solution en production.

- Réduire le temps entre une idée et sa mise en production

Dans un modèle classique, une idée traverse une chaîne longue : développement, tests manuels, validation, mise en production planifiée.
Avec DevOps, cette chaîne est en grande partie automatisée. Le passage d'une étape à l'autre devient fluide, presque mécanique.

Comment c'est rendu possible ?

- ✓ L'automatisation

On automatise les tests, les déploiements, les vérifications. Moins d'intervention humaine répétitive = moins d'erreurs et plus de rapidité.

- ✓ L'intégration continue (CI)

Chaque fois qu'un développeur ajoute du code, il est immédiatement intégré au reste du projet et testé automatiquement. On détecte les problèmes très tôt, avant qu'ils ne deviennent monstrueux.

- ✓ Les pipelines CI/CD

Un pipeline, c'est une chaîne d'étapes automatiques : compilation, tests, analyse, déploiement.
CI signifie Continuous Integration (intégration continue).
CD signifie Continuous Delivery ou Continuous Deployment (livraison ou déploiement continu).
En clair : dès que le code est prêt et validé, il peut aller en production presque sans friction.

✓ La standardisation des environnements

Un vieux cauchemar du développement : “ça marche sur ma machine”.
DevOps réduit ce problème en standardisant les environnements (mêmes configurations entre développement, test et production).
Moins de différences = moins de surprises.

En résumé, DevOps transforme le déploiement d’un événement stressant en routine banale.

Et ça touche à une idée plus large : en science comme en ingénierie, raccourcir les cycles d’expérimentation accélère l’apprentissage. Plus on peut tester vite, plus on progresse vite. Les entreprises qui maîtrisent ça ne vont pas seulement plus vite — elles apprennent plus vite. Et dans un monde complexe, la vitesse d’apprentissage est souvent plus stratégique que la vitesse brute.

c. amélioration continue

DevOps n’est pas un “projet” qu’on termine un vendredi soir avec un PowerPoint triomphant. Ce n’est pas un bouton qu’on active. C’est une dynamique permanente.

Amélioration continue signifie que le système lui-même apprend.

✓ Mesurer ce qui fonctionne

On ne peut pas améliorer ce qu’on ne mesure pas. Temps de déploiement, fréquence des incidents, durée des pannes, satisfaction utilisateur… Ces indicateurs deviennent des instruments de navigation.
Sans mesure, on avance à l’instinct. Avec des mesures, on avance avec des données.

✓ Identifier les points faibles

Les incidents ne sont pas vus comme des échecs personnels, mais comme des sources d’information.
Une panne devient une expérience : qu’est-ce qui a cassé ? Pourquoi ? Comment éviter que ça se reproduise ?
On parle souvent de “post-mortem” sans recherche de coupable. Le but est d’améliorer le système, pas d’humilier quelqu’un.

✓ Corriger

Une fois le problème identifié, on agit. Pas dans six mois. Rapidement.

La correction fait partie du flux normal de travail. Elle n'est pas perçue comme une interruption, mais comme un investissement.

- ✓ Automatiser

Si un problème revient plusieurs fois, la question devient : pourquoi dépend-on encore d'une action manuelle ?
L'automatisation est une manière d'intégrer la leçon dans le système lui-même.
C'est un peu comme passer de "je fais attention à ne pas me brûler" à "j'installe un thermostat intelligent".

- ✓ Améliorer progressivement

Il n'y a pas de perfection finale. On cherche des gains incrémentaux. 1 % d'amélioration ici, 2 % là.
Additionnées sur le long terme, ces petites optimisations changent radicalement la performance globale. C'est le même principe que l'évolution biologique : petites variations, grande transformation.

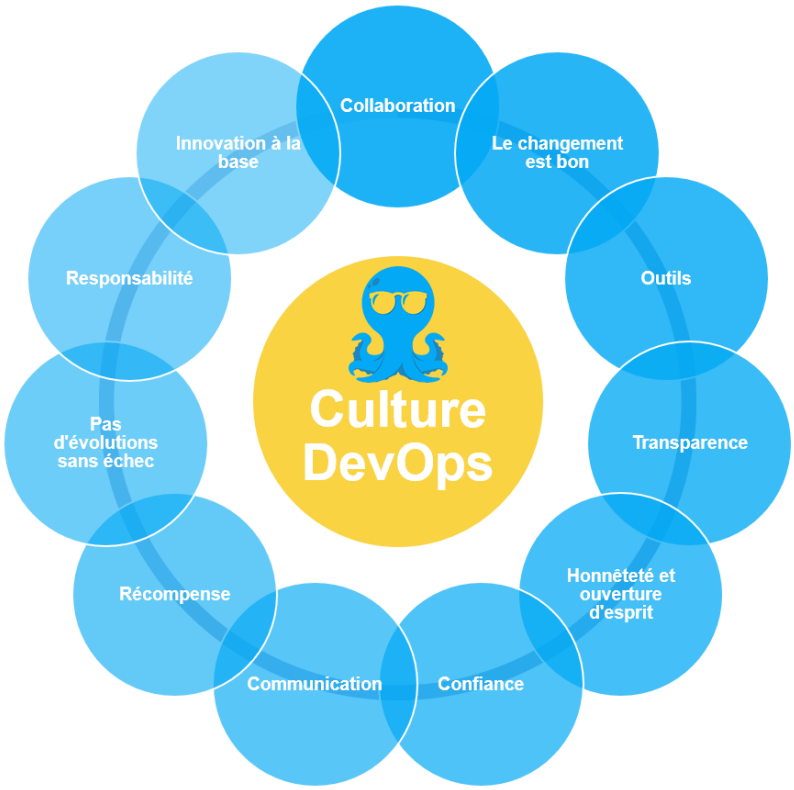
Ce qui est fascinant, c'est que DevOps applique au monde de l'entreprise une logique très proche de la méthode scientifique.
On observe.
On mesure.
On formule une hypothèse d'amélioration.
On teste.
On ajuste.

Ce n'est pas spectaculaire. Ce n'est pas héroïque. C'est **méthodique**.

Et paradoxalement, les organisations qui adoptent cette humilité permanente deviennent souvent les plus innovantes. Parce qu'elles ne cherchent pas à avoir raison une fois pour toutes — elles cherchent à apprendre en continu. Dans un environnement complexe, la capacité d'adaptation dépasse largement la planification parfaite.

Principes de Base de DevOps

1. La culture DevOps : ce que cela signifie réellement



a. Les valeurs fondamentales de DevOps

La culture DevOps repose sur quelques valeurs essentielles :

- **Collaboration** : Dev et Ops travaillent ensemble dès le début, au lieu de se passer le relais à la fin.
- **Communication** : les informations circulent de manière claire et rapide, notamment en cas de problème.
- **Responsabilité partagée** : la qualité, la performance et la disponibilité d'une application concernent tout le monde.

- **Automatisation** : tout ce qui est répétitif doit être automatisé pour réduire les erreurs humaines.
- **Feedback rapide** : les retours (tests, utilisateurs, monitoring) doivent arriver vite pour corriger tôt.
- **Amélioration continue** : on apprend des erreurs et on améliore le système en permanence.

Ces valeurs sont souvent résumées dans des modèles connus comme **CALMS** (Culture, Automation, Lean, Measurement, Sharing), qui structure l'esprit DevOps.

b. Collaboration et communication : un changement de posture

Dans un modèle classique, le cycle est souvent :

Dev développe → Ops déploie → Ops subit → Dev corrige

Ce modèle crée :

- des tensions,
- des retards,
- et des déploiements risqués.

Avec DevOps, l'idée est plutôt :

Dev + Ops conçoivent → automatisent → testent → déploient → surveillent → améliorent

Cela implique :

- des échanges réguliers,
- des outils partagés,
- et des objectifs communs.

c. L’impact sur les processus et les performances

Quand DevOps est réellement intégré — pas juste affiché sur une slide — les processus deviennent plus robustes et les performances s’améliorent de façon mesurable.

- Moins d’erreurs lors des déploiements

Pourquoi ? Parce que les déploiements sont automatisés, testés en continu et effectués plus fréquemment.

Un déploiement rare est un événement risqué. Un déploiement quotidien devient une routine maîtrisée.

La répétition réduit l’improvisation. L’automatisation réduit l’erreur humaine.

- Réduction du temps de mise en production

On parle souvent de “time to market” : le temps entre l’idée et sa disponibilité pour l’utilisateur.

Avec DevOps, les validations sont intégrées au flux. Les tests ne sont plus un barrage final, mais un contrôle permanent.

Résultat : moins d’attente, moins de blocages, plus de fluidité.

- Meilleure stabilité des applications

Cela peut sembler paradoxal. Livrer plus souvent pourrait sembler plus risqué. En réalité, c’est l’inverse.

De petites modifications sont plus faciles à comprendre, tester et isoler.

Quand un problème apparaît, il est plus simple d’en identifier la cause.

La stabilité vient d’une meilleure maîtrise du changement.

- Meilleure réactivité en cas d’incident

Une organisation DevOps surveille en permanence ses systèmes (monitoring).

Un incident n’est pas découvert par hasard, mais détecté rapidement.

Les équipes savent quoi faire, car les processus sont documentés et testés.

On réduit le “temps moyen de réparation”, ce qui est un indicateur clé de maturité opérationnelle.

- Meilleure satisfaction utilisateur

Au final, l’utilisateur ne voit pas “DevOps”.

Il voit une application qui fonctionne, qui évolue régulièrement, et dont les problèmes sont corrigés vite.

La technique devient invisible — et c’est souvent le signe qu’elle est bien maîtrisée.

Et surtout, il y a un effet humain majeur : la baisse du stress.

Dans un modèle classique, les mises en production sont vécues comme des moments à haute tension.

Dans une culture DevOps mature, la livraison devient un processus normalisé, presque banal.

On passe d’un modèle héroïque — où l’on sauve la situation à 3 heures du matin — à un modèle systémique, où l’on conçoit pour éviter le chaos.

C’est une transformation subtile mais profonde : au lieu de dépendre d’exploits individuels, on construit un système fiable. Et dans les systèmes complexes, ce sont les processus solides — pas les coups d’éclat — qui font la différence durable.Et surtout :
DevOps permet de transformer la livraison logicielle en une activité **plus fluide, plus contrôlée, et moins stressante**.

2. Cycle de Vie DevOps Introduction aux Outils DevOps

a. Les grandes étapes du cycle DevOps

Le cycle DevOps peut être décrit par plusieurs étapes clés :

1. **Planification (Plan)**
   - On définit ce qu’on doit construire et pourquoi.
   - On choisit ce qui est prioritaire.
   - On organise le travail pour avancer de manière structurée.
2. **Développement (Code)**
   - On écrit le code des nouvelles fonctionnalités.
   - On enregistre les modifications avec un outil de versioning comme Git.
   - On collabore pour produire un code cohérent et maintenable.
3. **Intégration (Build / CI)**
   - On assemble automatiquement le projet.
   - Chaque modification est intégrée rapidement au reste du code.
   - Des vérifications automatiques se déclenchent à chaque changement.
4. **Tests (Test)**
   - On vérifie que chaque partie du code fonctionne correctement.
   - On détecte les erreurs le plus tôt possible.
   - On réduit le risque de bugs en production.
5. **Déploiement (Deploy / CD)**
   - On met à disposition la nouvelle version automatiquement ou presque.
   - On limite les manipulations manuelles risquées.
   - On livre plus souvent, avec moins de stress.
6. **Exploitation (Operate)**
   - On gère les serveurs et l’infrastructure.
   - On veille à la performance et à la disponibilité.
   - On intervient rapidement en cas de problème.

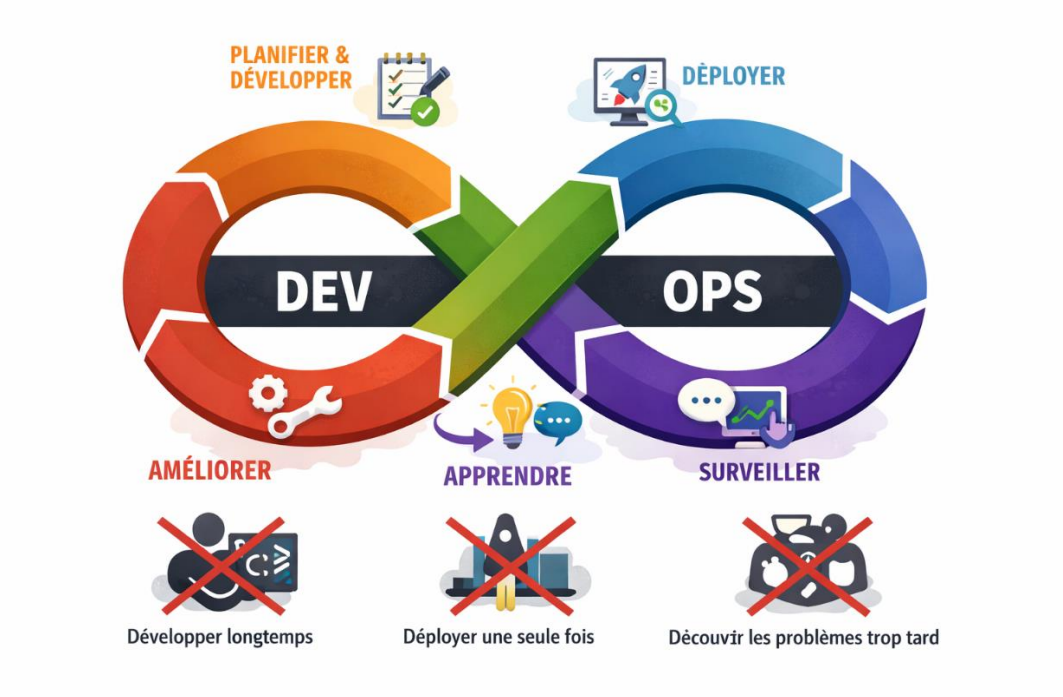
7. **Surveillance (Monitor)**
    - On observe le comportement de l'application en temps réel.
    - On analyse les journaux d'activité.
    - On reçoit des alertes en cas d'anomalie.
8. **Retour d'expérience (Feedback)**
    - On analyse les incidents pour comprendre leurs causes.
    - On écoute les retours des utilisateurs.
    - On améliore progressivement le produit et les pratiques.

Ce cycle ne s'arrête jamais. Il tourne en boucle.

Et plus il tourne vite — sans perdre en qualité — plus l'organisation apprend.

DevOps, au fond, c'est ça : transformer le développement logiciel en un système vivant qui s'améliore à chaque respiration.

b. Une boucle continue, pas une ligne droite



Le point central de DevOps est que ces étapes forment une boucle :

- on développe,
- on déploie,
- on surveille,
- on apprend,
- puis on améliore.

Cela permet d'éviter le modèle où :

- on développe pendant longtemps,
- puis on déploie une seule fois,
- puis on découvre tous les problèmes trop tard.

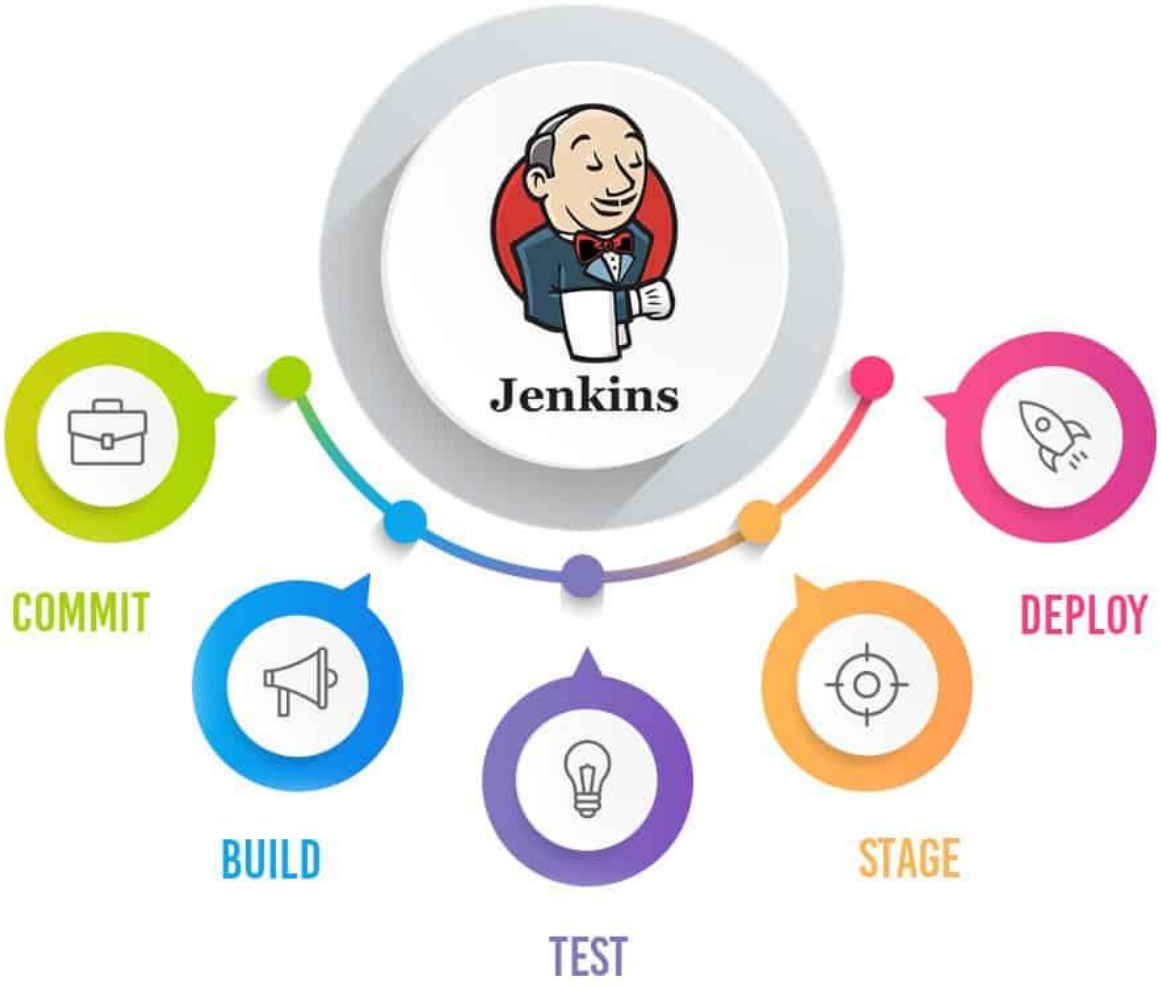
c. Ce que DevOps change réellement dans le cycle de vie

DevOps permet de rendre le cycle :

- **plus court** (livraison plus rapide),
- **plus fiable** (automatisation + tests),
- **plus transparent** (monitoring + feedback),
- et **plus robuste** (réduction des erreurs humaines).

Introduction aux Outils DevOps

1. Jenkins pour CI/CD



a. Pourquoi Jenkins (et pourquoi la CI/CD) ?

Dans un environnement classique, les étapes de développement sont souvent manuelles :

- un développeur compile sur sa machine,
- exécute des tests localement,

- puis envoie le code,
- puis une autre personne essaye de déployer.

Ce fonctionnement provoque plusieurs problèmes :

- les tests ne sont pas exécutés systématiquement,
- les erreurs arrivent tard,
- les livraisons sont lentes,
- et la mise en production devient stressante.

☞ La CI/CD (Intégration Continue / Livraison Continue) répond à ce problème.

b. Définition : Intégration Continue (CI)

L'intégration continue consiste à :

- intégrer le code régulièrement dans un dépôt commun (ex : Git),
- déclencher automatiquement des étapes à chaque modification,
- vérifier que le projet reste fonctionnel.

Dans un pipeline CI, on retrouve souvent :

- récupération du code,
- compilation,
- exécution de tests,
- vérification de qualité.

Objectif : détecter les erreurs le plus tôt possible.

c. Définition : Livraison Continue (CD)

La livraison continue consiste à automatiser la préparation au déploiement :

- packaging,
- création d'un artefact (ex : un fichier .jar, un .zip, ou une image Docker),
- publication de l'artefact,
- déploiement sur un environnement de test ou staging.

📌 La différence importante :

- **Continuous Delivery** : le logiciel est prêt à être déployé, mais la décision peut rester humaine.
- **Continuous Deployment** : le déploiement se fait automatiquement après validation.

d. Jenkins : rôle et fonctionnement

Jenkins est un outil d'autatisation utilisé pour construire des pipelines.

Un pipeline Jenkins sert à :

- exécuter des tâches automatiquement,
- standardiser les étapes,
- garantir la reproductibilité,
- fournir un historique des exécutions.

Dans DevOps, Jenkins devient un "chef d'orchestre" qui automatise la chaîne :

code → build → tests → packaging → déploiement

e. Notion de pipeline

Un pipeline Jenkins est une succession d'étapes (stages).

Exemple typique :

- **Checkout** : récupérer le code depuis Git
- **Build** : compiler / assembler l'application
- **Test** : exécuter les tests
- **Package** : produire un artefact
- **Deploy** : déployer sur un environnement

f. Bonnes pratiques CI/CD avec Jenkins

Un point très important : en DevOps, un pipeline doit être :

- **reproductible**
- **versionné**
- **maintenable**
- **sécurisé**

Donc on recommande :

➢ Pipeline as Code

Le pipeline est défini dans un fichier du projet (ex : Jenkinsfile) versionné dans Git.

➢ Tests automatiques

Les tests doivent être systématiques.
Une pipeline sans tests = un “déploiement automatique de bugs”.

➢ Gestion des secrets

Les mots de passe et clés ne doivent jamais être écrits en clair dans le pipeline.

➢ Feedback rapide

Le pipeline doit fournir rapidement :

- succès/échec,
- logs,
- rapports.

2. Docker et la gestion des conteneurs

a. Pourquoi Docker est devenu central en DevOps ?

L'un des problèmes historiques du développement est :

"ça marche sur ma machine"

Ce problème vient des différences entre environnements :

- versions différentes,
- dépendances manquantes,
- configurations incohérentes,
- OS différent,
- variables d'environnement non identiques.

Docker résout ce problème en emballant l'application + ses dépendances dans un conteneur.

b. Conteneur vs Machine virtuelle (VM)

Une confusion fréquente :

- Une **VM** embarque un OS complet (plus lourd)
- Un **conteneur** partage le noyau du système et embarque seulement le nécessaire

📌 Résultat :

- conteneurs plus légers,
- plus rapides à lancer,
- plus faciles à déployer à grande échelle.

c. Concepts fondamentaux Docker

➢ Image Docker

Une image est un modèle immuable, construit à partir d'un Dockerfile.

➢ Conteneur Docker

Un conteneur est une instance en exécution d'une image.

➢ Dockerfile

Un fichier décrivant comment construire l'image :

- base (ex : python, node, ubuntu…)
- copie du code
- installation des dépendances
- commande de lancement

➢ Registry

Lieu où stocker les images :

- Docker Hub
- GitHub Container Registry
- registry privé d'entreprise

d. Avantages Docker dans DevOps

Docker apporte des bénéfices majeurs :

- **reproductibilité** : même environnement partout
- **portabilité** : fonctionne sur dev, test, prod
- **déploiement rapide**
- **scalabilité** : plus facile à dupliquer
- **isolation** : chaque service dans son conteneur

e. Docker dans un pipeline DevOps

Docker s'intègre parfaitement dans CI/CD :

- Jenkins compile et teste
- Jenkins construit une image Docker
- Jenkins pousse l'image vers un registry
- Jenkins déclenche le déploiement

☞ Docker devient donc le format standard de livraison.

3. Kubernetes pour l'orchestration des conteneurs

a. Pourquoi Kubernetes ?

Docker permet de lancer des conteneurs.
Mais en entreprise, on a des besoins supplémentaires :

- redémarrer automatiquement un conteneur tombé,
- répartir les conteneurs sur plusieurs machines,
- gérer la montée en charge,
- faire des mises à jour progressives,
- assurer la disponibilité.

☞ Kubernetes automatise tout cela.

b. Concepts clés Kubernetes

➢ Cluster

Ensemble de machines gérées par Kubernetes.

➢ Node

Une machine du cluster.

➢ Pod

Un pod contient un ou plusieurs conteneurs.

➢ Deployment

Objet Kubernetes qui gère :

- combien de pods doivent tourner,
- comment les mettre à jour,
- comment les remplacer en cas de panne.

➢ Service

Expose un ensemble de pods avec une adresse stable.

➢ kubectl

Outil de commande pour interagir avec Kubernetes.

c. Déploiement d'une application

Déployer une application sur Kubernetes consiste à :

- fournir un manifeste YAML,
- définir l'image Docker,
- définir le nombre de réplicas,
- définir l'exposition réseau.

Kubernetes se charge ensuite de :

- créer les pods,
- les maintenir en vie,
- gérer les mises à jour.

d. Pourquoi Kubernetes est un outil DevOps clé ?

Kubernetes est central en DevOps car il :

- standardise le déploiement
- automatise la résilience
- facilite le scaling
- permet une exploitation plus moderne

4. Ansible pour la gestion des configurations

a. Le problème : la configuration manuelle

Dans beaucoup d'organisations, la configuration des serveurs est faite :

- à la main,
- par des commandes copiées-collées,
- sans documentation,
- et sans traçabilité.

Cela crée :

- des environnements incohérents,
- des erreurs humaines,
- des serveurs "magiques" qu'une seule personne sait gérer.

b. Ansible : définition

Ansible est un outil d'automatisation permettant de :

- configurer des machines,
- installer des logiciels,

- déployer des applications,
- via des playbooks en YAML.

c. Concepts fondamentaux Ansible

➢ Inventaire

Liste des machines à gérer.

➢ Modules

Actions prêtes à l'emploi (installer un paquet, créer un utilisateur…).

➢ Playbook

Suite d'instructions.

➢ Idempotence

Exécuter plusieurs fois le playbook doit produire le même résultat sans casser le système.

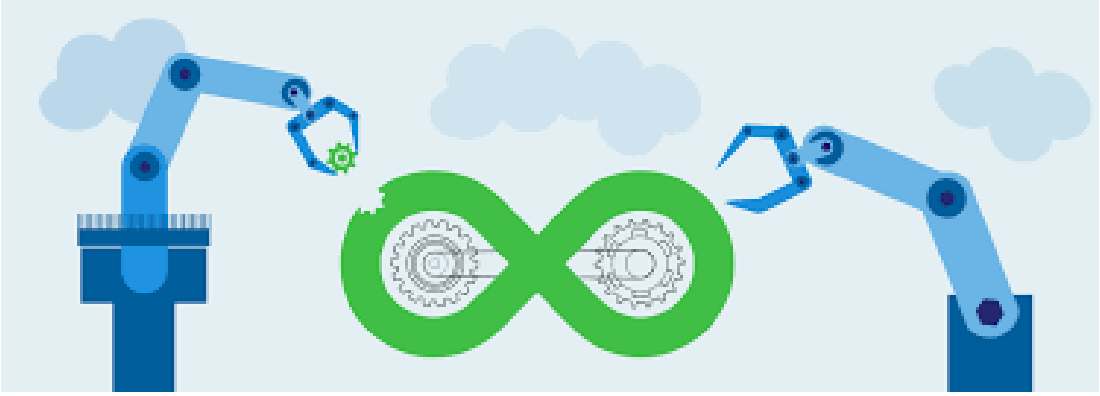
d. Ansible dans une démarche DevOps

Ansible permet :

- d'automatiser l'exploitation,
- de rendre l'infrastructure reproductible,
- de réduire les erreurs humaines.

Ansible peut aussi être déclenché par Jenkins dans un pipeline CI/CD.

Automatisation des Processus DevOps



Partie 1 - Scripts d'automatisation & Infrastructure as Code (Terraform)

1. L'automatisation : le pilier technique de DevOps

Dans DevOps, l'automatisation est aussi importante que la collaboration.

Pourquoi ?

Parce que DevOps cherche à réduire les "zones de risque" dans un projet logiciel.
Or, une grande partie des risques vient du manuel.

Exemple simple

Imaginons un déploiement manuel :

- Copier le code sur le serveur
- Installer les dépendances
- Modifier une configuration
- Redémarrer le service

- Vérifier que ça marche

Même si ce processus paraît simple, il devient vite problématique :

- une étape peut être oubliée,
- l'ordre peut être inversé,
- la version installée peut être différente,
- une dépendance peut manquer,
- et le résultat peut varier d'un serveur à un autre.

📌 Et plus on déploie souvent, plus ces risques se multiplient.

2. Pourquoi les tâches manuelles deviennent un problème à grande échelle

Au début d'un projet, les tâches manuelles semblent "acceptables".

Mais dès que le projet grandit, on rencontre plusieurs difficultés :

a. Incohérence entre environnements

Le serveur de test n'est pas configuré comme la production.
Donc un bug peut apparaître uniquement en production.

b. Dépendance à une personne

Une seule personne connaît "la procédure".

Si cette personne est absente :

- personne ne déploie,
- personne ne corrige,
- le projet est bloqué.

c. Manque de traçabilité

Si une panne arrive, on ne sait pas :

- ce qui a été modifié,
- par qui,
- quand,
- et pourquoi.

d. Stress lors des mises en production

Plus un déploiement est manuel, plus il est stressant.

C'est d'ailleurs l'une des raisons historiques de la création de DevOps :
réduire le stress et l'incertitude autour de la production.

3. Scripts DevOps : Bash et Python

Les scripts sont souvent la première étape vers l'automatisation DevOps.

Ils permettent de transformer une série d'actions répétitives en une commande unique.

a. Pourquoi utiliser des scripts ?

Un script sert à :

- automatiser une action répétée,
- garantir que l'action est faite dans le bon ordre,
- éviter l'oubli d'étapes,
- et produire un résultat constant.

b. Exemples de tâches automatisables

Un script peut automatiser :

- la compilation d'un projet,
- le lancement des tests,
- la création d'un package,
- l'installation de dépendances,
- le nettoyage de fichiers,
- une sauvegarde automatique,

- ou même une vérification de santé (health check).

En DevOps, tout ce qui est répétitif devient un candidat naturel à l'automatisation.

---

4. Bash vs Python : quand utiliser quoi ?

Bash

Bash est particulièrement utile pour :

- enchaîner des commandes Linux,
- gérer des fichiers,
- automatiser des actions système,
- manipuler des services.

📌 Bash est souvent utilisé par les administrateurs système et les DevOps.

Python

Python est souvent utilisé lorsque le script devient plus complexe :

- manipulation de données,
- interaction avec des APIs (ex : GitLab, Jenkins, Kubernetes),
- génération de rapports,
- automatisation plus avancée.

📌 Python est aussi plus lisible et plus maintenable sur de gros scripts.

5. Bonnes pratiques essentielles pour les scripts DevOps

Un script DevOps doit être fiable.

Parce qu'en DevOps, les scripts finissent souvent par être exécutés :

- automatiquement,
- régulièrement,

- parfois en production.

Donc un script fragile peut causer des incidents.

Bonnes pratiques recommandées :

a. Produire des logs

Un script doit expliquer ce qu'il fait.

Sinon, en cas de problème, on ne comprend rien.

b. Gérer les erreurs

Si une étape critique échoue, le script doit s'arrêter.

Continuer malgré une erreur = produire une situation incohérente.

c. Éviter les valeurs en dur

Utiliser des variables d'environnement ou des fichiers de configuration.

d. Ne jamais stocker des secrets en clair

Pas de mots de passe dans le script.

6. Infrastructure as Code (IaC) : automatiser l'infrastructure

Une idée essentielle à comprendre :

DevOps ne concerne pas uniquement le déploiement du code.
DevOps concerne aussi l'infrastructure.

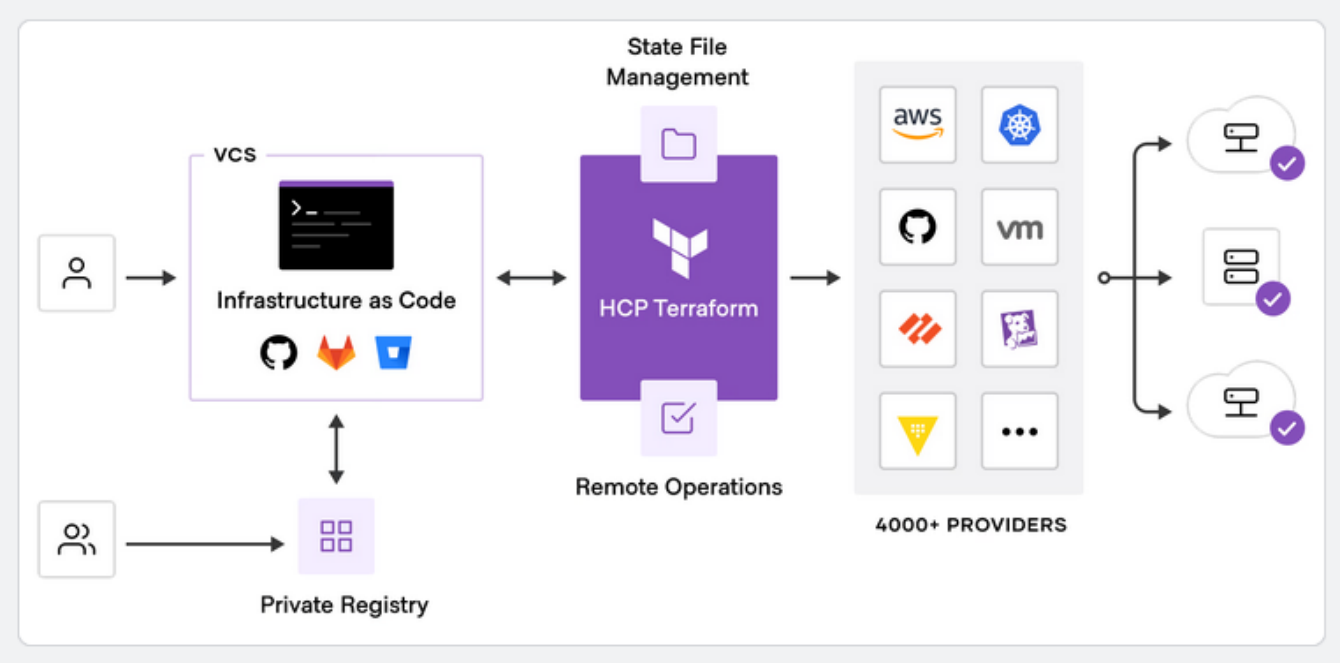
Dans un système moderne, l'infrastructure peut inclure :

- serveurs,
- réseau,
- règles de sécurité,
- stockage,
- bases de données,
- load balancers,
- etc.

Si tout cela est fait manuellement, on retombe dans les mêmes problèmes :

- incohérence,
- lenteur,
- erreurs humaines,
- impossibilité de reproduire.

7. Terraform : introduction et logique de fonctionnement



Terraform est un outil qui permet de décrire l'infrastructure sous forme de code.

a. Terraform : la logique déclarative

Terraform fonctionne sur une idée très importante :

- On ne décrit pas "les commandes à exécuter".
- On décrit "l'état final souhaité".

Exemple :
Au lieu de dire :

"Crée une machine, puis ouvre le port, puis installe…"

Terraform dit :

"Je veux une machine avec telle configuration, et un réseau avec telles règles."

Terraform calcule ensuite ce qu'il doit faire pour atteindre cet état.

b. Concepts clés Terraform

❖ **Provider**

Le provider est le connecteur vers une plateforme :

- AWS, Azure, GCP,
- VMware,
- ou d'autres systèmes.

❖ **Ressources**

Les ressources sont les objets à créer :

- une machine virtuelle,
- un réseau,
- un firewall,
- un stockage.

❖ **State (état)**

Terraform garde une mémoire de ce qu'il a créé.

Ce fichier est essentiel car il permet de :

- comparer l'état souhaité et l'état réel,
- appliquer les modifications proprement.

❖ **Plan / Apply**

Terraform propose une démarche très DevOps :

- **plan** : on simule ce qui va changer
- **apply** : on applique réellement

📌 Cela réduit énormément les erreurs, car on voit l'impact avant d'agir.

8. Ce que Terraform apporte concrètement en DevOps

Terraform permet de :

- recréer un environnement en quelques minutes,
- cloner une infrastructure pour une autre équipe,
- standardiser les environnements (dev/test/prod),
- documenter l'infrastructure via le code,
- travailler en équipe sur l'infrastructure via Git.

Terraform transforme donc l'infrastructure en un objet :

- versionné,
- traçable,
- reproductible.

Ce qui correspond parfaitement à la philosophie DevOps.

Partie 2 - Orchestration avancée Kubernetes & Déploiement automatisé Ansible

1. Kubernetes avancé : pourquoi c'est important ?

Dans un projet simple, on peut lancer un conteneur Docker sur une machine.

Mais dès que l'application devient sérieuse, on a des besoins réels :

- l'application doit être disponible en continu,
- les mises à jour doivent être possibles sans interruption,
- la charge peut varier,
- des pannes peuvent survenir.

Kubernetes existe pour gérer ces problèmes automatiquement.

2. Orchestration : ce que Kubernetes fait réellement

Kubernetes ne se contente pas de "lancer" un conteneur.

Il assure que :

- le bon nombre de conteneurs tourne,
- les conteneurs redémarrent en cas de panne,
- l'application est mise à jour progressivement,
- la charge est répartie.

C'est exactement la logique DevOps :

automatiser la stabilité et la fiabilité

3. Notions avancées Kubernetes (bien expliquées)

a. Rolling update (mise à jour progressive)

Un déploiement classique peut être brutal :

- on coupe l'ancienne version,
- on démarre la nouvelle version.

Résultat : coupure possible.

Kubernetes permet de faire mieux :

- il démarre progressivement des pods de la nouvelle version,
- puis retire progressivement l'ancienne version.

📌 Résultat : moins (voire aucune) interruption.

b. Rollback (retour arrière)

Une mise à jour peut échouer.

Kubernetes permet de revenir à une version précédente rapidement.

C'est une notion très importante en DevOps :

On peut livrer souvent seulement si l'on peut revenir en arrière rapidement.

c. Health checks : liveness et readiness

Un conteneur peut tourner mais être "bloqué".

Donc Kubernetes vérifie la santé.

❖ Readiness probe

Vérifie si l'application est prête à recevoir du trafic.

❖ Liveness probe

Vérifie si l'application est vivante ou bloquée.

Ces sondes rendent le système plus robuste.

d. Scaling (mise à l'échelle)

Scaling = augmenter le nombre de pods.

Cela permet :

- de gérer plus d'utilisateurs,
- de répartir la charge.

Kubernetes permet :

- scaling manuel (modifier le nombre de réplicas)
- autoscaling (ajustement automatique selon métriques)

e. Gestion des défaillances

Kubernetes surveille l'état souhaité.

Donc si :

- un pod tombe → Kubernetes le recrée
- une machine tombe → Kubernetes replanifie ailleurs (selon configuration)

📌 Résultat : le système est plus résilient.

4. Ansible : automatiser le déploiement dans la vraie vie

Enormément de personnes pensent que déployer = "copier le code".

En réalité, un déploiement réel inclut :

- installation des dépendances,
- configuration des services,
- variables d'environnement,
- gestion des ports,
- redémarrage contrôlé,
- logs.

Si tout cela est manuel, alors :

- le déploiement est fragile,
- il varie d'une machine à une autre,
- il est difficile à reproduire.

Ansible résout cela en décrivant la procédure sous forme de playbook.

---

5. Ansible avancé : structurer pour être maintenable

Quand les playbooks grandissent, il faut structurer.

Ansible propose :

a. Rôles

Permet de découper un déploiement en blocs réutilisables.

b. Variables

Permet d'adapter :

- dev
- test
- prod

sans réécrire tout.

c. Templates

Permet de générer automatiquement des fichiers de configuration.

d. Handlers

Permet de redémarrer un service seulement si nécessaire.

📌 C’est très important, car cela rend le déploiement plus stable.

6. Vision DevOps : connecter Jenkins, Docker, Kubernetes, Ansible

Cette partie de ce cours permet aussi de comprendre la chaîne globale :

- Jenkins construit et teste
- Docker produit une image reproductible
- Kubernetes orchestre l’exécution
- Ansible automatise la configuration et le déploiement

DevOps n’est donc pas une liste d’outils.

C’est une architecture de livraison logicielle.

Synthèse générale — Automatisation DevOps

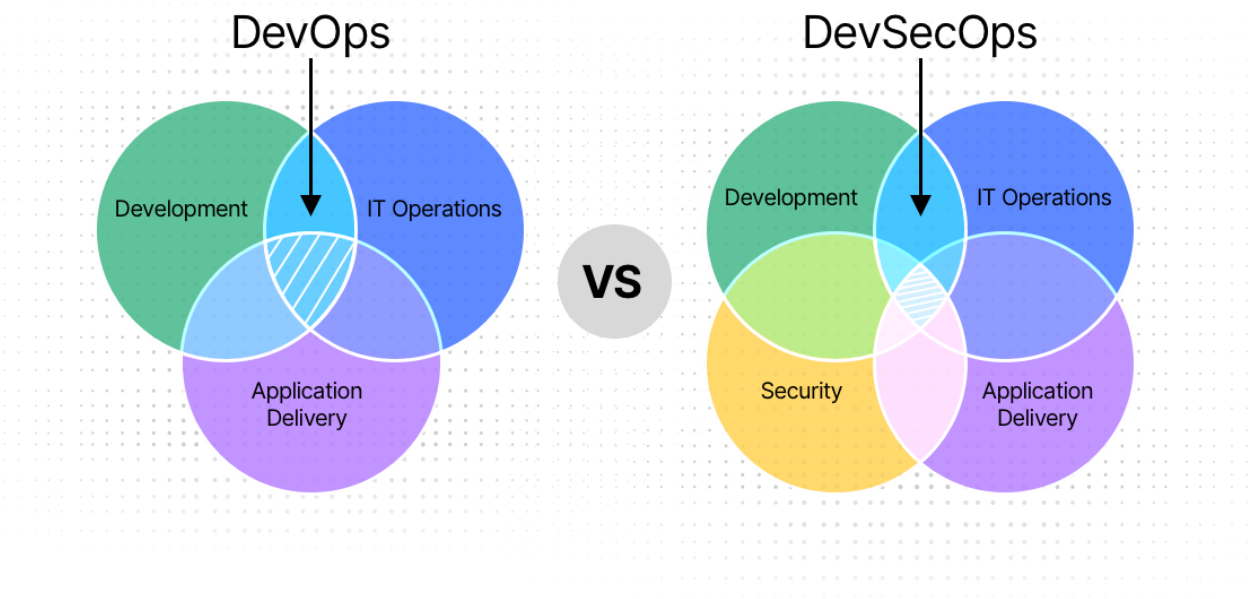
DevOps devient réellement puissant lorsque tout ce qui est répétitif est automatisé.

Et chaque outil joue un rôle :

- **Scripts** : automatisation simple et rapide
- **Terraform** : automatisation de l'infrastructure
- **Kubernetes** : automatisation de l'exécution et de la résilience
- **Ansible** : automatisation de la configuration et du déploiement

Sécurité dans DevOps

Partie 1 - DevSecOps : intégrer la sécurité dès le début & outils de sécurité



1. Pourquoi la sécurité est devenue un enjeu central en DevOps

- Dans le passé, beaucoup d'organisations fonctionnaient ainsi :
- On développe l'application
- On la teste
- On la déploie
- À la fin… on demande à l'équipe sécurité de vérifier

Ce modèle pose un énorme problème.

Pourquoi ?

Parce que lorsqu'une faille est découverte à la fin :

- il faut revenir en arrière,
- modifier du code,
- parfois modifier l'architecture,
- retarder la mise en production,
- et mobiliser plusieurs équipes.

📌 Résultat :
La sécurité devient perçue comme un "frein", alors qu'elle devrait être une protection.

2. Le problème réel : la sécurité arrive trop tard

Dans un projet réel, les failles ne sont pas rares.

Elles peuvent venir de :

- mauvaises pratiques de développement,
- dépendances vulnérables,
- erreurs de configuration,
- secrets exposés,
- ou erreurs humaines.

Et si on ne les détecte qu'en production :

- l'impact est plus grave,
- les corrections coûtent plus cher,
- et les utilisateurs sont exposés.

3. DevSecOps : définition et philosophie

DevSecOps signifie :

Development + Security + Operations

L'idée est très simple :

- DevOps vise la rapidité et la fiabilité
- DevSecOps ajoute : **la sécurité doit être intégrée au même niveau**

Donc DevSecOps cherche à :

- automatiser des contrôles de sécurité,
- intégrer la sécurité dans les pipelines,
- et rendre la sécurité continue.

4. Le principe “Shift Left” : déplacer la sécurité vers la gauche

Dans un cycle DevOps, on peut représenter les étapes :

- Plan → Code → Build → Test → Deploy → Operate → Monitor → Feedback

Traditionnellement, la sécurité était plutôt à droite (vers la fin).

DevSecOps dit :

On déplace la sécurité vers la gauche : dès le code, dès le build, dès les tests.

Pourquoi c’est une bonne idée ?

Parce que plus une erreur est détectée tôt :

- ✓ plus elle est facile à corriger,
- ✓ moins elle coûte cher,
- ✓ moins elle a d’impact.

5. Quels sont les risques typiques dans un environnement DevOps ?

DevSecOps ne se limite pas à “protéger contre les hackers”.

Il vise surtout à réduire les vulnérabilités courantes, très fréquentes dans les projets.

a. Vulnérabilités dans les dépendances

Une application moderne utilise souvent :

- des bibliothèques (Python, Java, NodeJS…),
- des frameworks,
- des modules open source.

Le problème :

- certaines versions contiennent des failles,
- et les développeurs ne le savent pas toujours.

📌 Une application peut donc être vulnérable sans que le code écrit par l'équipe soit "mauvais".

b. Mauvaises configurations

Une grande partie des incidents de sécurité vient de configurations incorrectes :

- port ouvert inutilement,
- permissions trop larges,
- stockage accessible publiquement,
- règles firewall mal définies.

c. Secrets exposés

Un des risques les plus dangereux :

- mot de passe dans un fichier,
- clé API commitée dans Git,
- token Jenkins dans un script.

Ce problème arrive souvent par erreur humaine.

d. Failles applicatives classiques

Exemples :

- injection SQL,
- XSS,
- authentification faible,
- mauvaise gestion des sessions.

6. Outils de sécurité dans DevOps : comprendre les familles d'outils

L'objectif ici n'est pas de maîtriser tous les outils, mais de comprendre leur rôle dans un pipeline DevOps.

a. Analyse statique (SAST)

SAST = Static Application Security Testing

C'est une analyse du code source, sans exécuter l'application.

Cela permet de détecter :

- mauvaises pratiques,
- vulnérabilités potentielles,
- erreurs logiques,
- code dangereux.

📌 Exemple dans le cours : **SonarQube**

SonarQube analyse :

- qualité du code (maintenabilité),
- bugs,
- vulnérabilités,
- dette technique.

b. Analyse dynamique (DAST)

DAST = Dynamic Application Security Testing

Cette analyse se fait sur l'application en fonctionnement.

L'outil agit comme un attaquant simulé :

- il envoie des requêtes,
- teste des injections,
- cherche des failles.

📌 Exemple dans le cours : **OWASP ZAP**

OWASP ZAP permet de détecter :

- failles web,
- mauvaises protections,
- formulaires vulnérables,
- attaques classiques.

c. Pourquoi ces outils sont importants en DevOps ?

Parce qu'ils peuvent être :

- automatisés,
- exécutés à chaque commit,
- intégrés dans Jenkins.

Donc la sécurité devient :

- continue,
- mesurable,
- et intégrée au cycle de livraison.

7. Intégration des outils de sécurité dans Jenkins

Un pipeline DevSecOps typique peut inclure :

1. ***Build***
2. ***Tests unitaires***
3. ***Analyse qualité / sécurité (SonarQube)***
4. ***Scan dynamique (ZAP)***
5. ***Déploiement staging***
6. ***Déploiement production***

a. Notion de seuils (quality gate)

Un concept très important :

On peut définir des règles :

- si le nombre de vulnérabilités critiques dépasse un seuil → pipeline échoue

Cela évite le cas classique :

"On déploie quand même, on corrigera plus tard."

Partie 2 - Sécurité Kubernetes, gestion des accès, chiffrement & surveillance

1. Pourquoi Kubernetes demande une sécurité spécifique ?

Kubernetes est un orchestrateur.

Cela signifie qu'il contrôle :

- le déploiement,
- le réseau,
- les ressources,
- les secrets,
- les permissions.

Donc si un attaquant obtient un accès Kubernetes mal protégé :

- il peut accéder aux pods,
- lire des secrets,
- déployer des conteneurs malveillants,
- ou perturber tout le système.

📌 Kubernetes n'est pas "dangereux", mais il exige une bonne discipline de sécurité.

2. Sécurité Kubernetes : le principe du moindre privilège

Le principe du moindre privilège signifie :

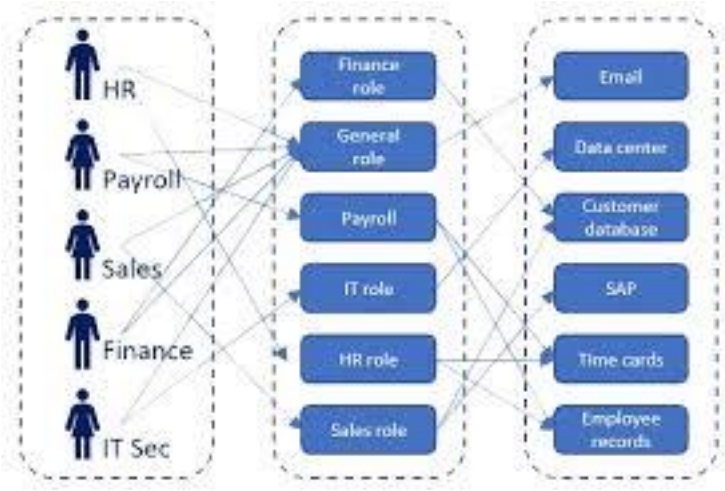
On donne à chaque utilisateur/service uniquement les droits nécessaires.

Exemple :

- un développeur n'a pas forcément besoin de supprimer des namespaces,

- une application n'a pas besoin de lire tous les secrets,
- un pipeline CI/CD n'a pas besoin d'être admin cluster.

3. RBAC : contrôle des accès par rôles



RBAC = Role-Based Access Control

C'est le mécanisme Kubernetes permettant de définir :

- qui a le droit de faire quoi,
- sur quelles ressources,
- dans quel namespace.

a. Pourquoi RBAC est indispensable ?

Sans RBAC :

- tout le monde devient admin,
- les erreurs humaines deviennent catastrophiques,
- un accès compromis donne trop de pouvoir.

RBAC permet donc de réduire fortement les risques.

b. Notions RBAC à comprendre

Même sans entrer dans la complexité complète, l'on doit comprendre :

- **Role** : permissions dans un namespace
- **ClusterRole** : permissions globales
- **RoleBinding** : associe un rôle à un utilisateur
- **ServiceAccount** : compte utilisé par une application

4. Politiques de sécurité Kubernetes

En entreprise, Kubernetes doit aussi être protégé contre :

- des conteneurs trop privilégiés,
- des images non fiables,
- des pods qui consomment trop de ressources.

Exemples de pratiques :

➢ **Empêcher les conteneurs privilégiés**

Un conteneur privilégié peut accéder à des ressources sensibles.

➢ **Limiter CPU/RAM**

Un pod peut bloquer un cluster en consommant tout.

➢ **Contrôler les images**

Ne pas autoriser des images inconnues ou non vérifiées.

5. Gestion des secrets

Kubernetes permet de stocker des secrets (passwords, tokens…).

Mais il faut comprendre un point important :

Un secret mal géré reste un secret exposé.

✓ ***Bonnes pratiques***

- ne jamais commit des secrets dans Git
- utiliser des outils dédiés (vault, secrets manager) si possible
- limiter l'accès via RBAC
- faire tourner les secrets (rotation)

6. Chiffrement et surveillance : sécurité + résilience

a. Chiffrement en transit

Les communications doivent être chiffrées :

- entre utilisateurs et application
- entre services

Exemple : HTTPS / TLS

b. Chiffrement au repos

Les données stockées doivent être chiffrées :

- bases de données,
- volumes,
- backups.

c. Surveillance et audit

Une bonne sécurité DevOps repose aussi sur la détection rapide.

Donc on doit surveiller :

- logs,
- métriques,
- comportements anormaux,
- tentatives d'accès.

7. Pourquoi DevOps + sécurité = un équilibre

Une erreur fréquente est de croire que la sécurité bloque DevOps.

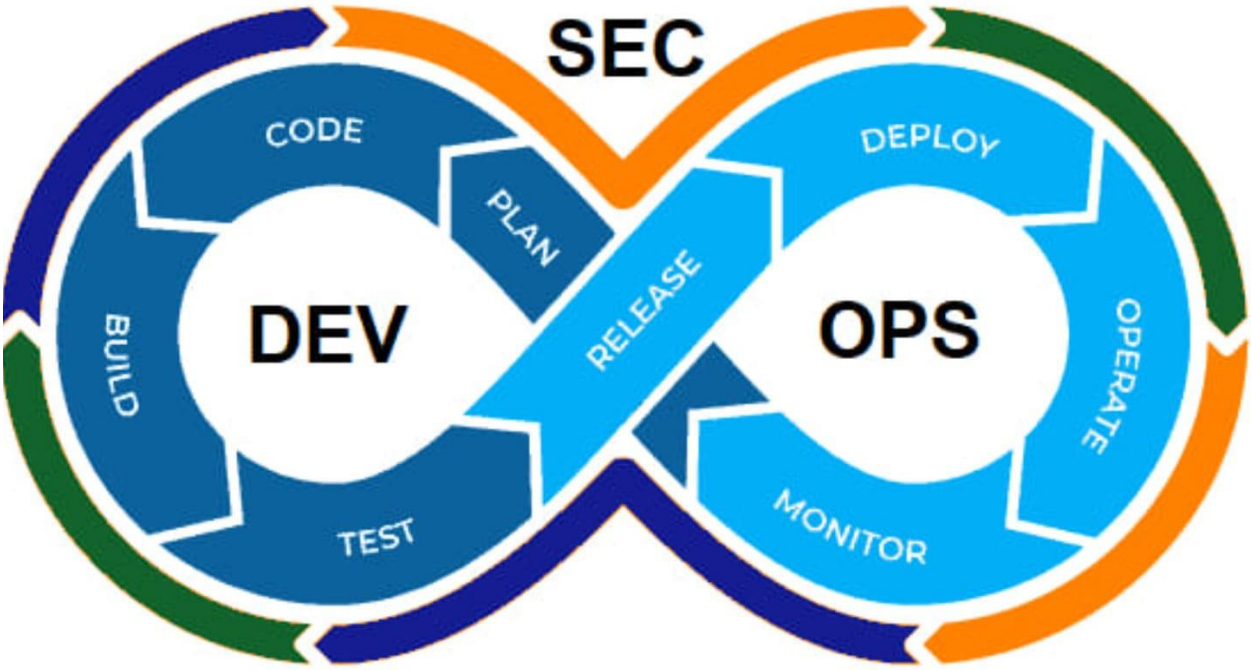
En réalité :

- DevOps veut aller vite
- La sécurité veut éviter les catastrophes

DevSecOps cherche donc un équilibre :

Automatiser la sécurité pour rester rapide sans devenir dangereux.

Synthèse générale — Sécurité dans DevOps



À la fin de ce volet, l'on retient cette synthèse très importante :

- DevSecOps = sécurité intégrée dans DevOps
- Shift-left = détecter tôt
- Jenkins peut automatiser les contrôles sécurité
- SonarQube (SAST) analyse le code
- OWASP ZAP (DAST) analyse l'application en fonctionnement
- Kubernetes exige RBAC et politiques de sécurité
- chiffrement + monitoring = sécurité moderne